

Laboratorio 7 - Entrega Final

Task 1: Análisis de DQN y Double DQN para detección de intrusiones

Task 2: Implementación y evaluación de DQN, Double DQN y PER

Task 1.1

a. Estado continuo de 256 dimensiones

DQN sí es arquitecturalmente apropiado. DQN puede recibir estados continuos; no exige imágenes ni estados discretos. El problema no es la cantidad de variables, sino que el vector esté bien normalizado y que las métricas relevantes estén presentes.

Usaríamos una red totalmente conectada (MLP): entrada de 256 valores, dos o tres capas densas con ReLU y una capa final de 4 salidas. Una CNN no es la primera opción porque el vector no tiene vecindad espacial como una imagen. Si los logs se entregaran como secuencias con orden temporal, se podría añadir una LSTM/GRU o atención antes de la MLP.

b. Espacio de acciones

DQN y sus variantes son apropiados porque hay solo 4 acciones discretas. La red puede calcular los cuatro valores $Q(s,a)$ en una sola pasada y escoger la mayor. DQN deja de ser práctico cuando las acciones son continuas o muy numerosas.

c. Intrusiones muy raras y Experience Replay

Sea $p < 0.001$ la proporción de transiciones críticas en un buffer de tamaño N . Con muestreo uniforme, el número esperado de transiciones críticas en un minibatch de tamaño B es:

$$E[K] = Bp < 0.001B.$$

Por ejemplo, con $B=64$, $E[K] < 0.064$. Además, la probabilidad de que un minibatch no contenga ninguna intrusión es:

$$P(K=0) = (1-p)^B \approx (0.999)^{64} = 0.938.$$

Por tanto, cerca del 94% de los minibatches no enseñan al agente qué hacer ante una intrusión real. El buffer queda dominado por tráfico normal y los gradientes favorecen decisiones frecuentes, como ignorar.

La variante indicada es **Prioritized Experience Replay (PER)** con DQN o Double DQN. PER asigna mayor probabilidad a transiciones con gran error TD, que suelen incluir eventos raros o mal aprendidos. Se deben usar pesos de importancia para reducir el sesgo que introduce el muestreo no uniforme. También conviene guardar y revisar explícitamente suficientes casos críticos; PER no sustituye la calidad ni la cobertura de los datos.

d. Recompensa y falsos positivos

Si se minimizan solo los falsos positivos, el agente puede aprender a ignorar casi todo. Eso baja las alertas equivocadas, pero aumenta los falsos negativos y deja pasar intrusiones reales.

Una recompensa posible por decisión es:

$$r = 100 \cdot 1[\text{intrusión detectada y contenida}] - 200 \cdot 1[\text{intrusión ignorada}] - 10 \cdot 1[\text{falso positivo}] - c(a).$$

Donde $c(a)$ representa el costo operativo: $c(\text{ignorar})=0$, $c(\text{alertar})=1$, $c(\text{bloquear})=4$ y $c(\text{aislar})=12$. Los componentes son: recompensa por detener una intrusión, castigo por falso negativo, castigo por falso positivo y costo de interrupción de cada acción.

El falso negativo recibe la mayor penalización porque una intrusión no contenida puede causar pérdida de datos o caída del servicio. El falso positivo se penaliza menos, pero lo suficiente para no bloquear tráfico normal. Aislar cuesta más que alertar o bloquear porque puede afectar a usuarios legítimos. Los valores exactos deben calibrarse con costos reales, pruebas históricas y límites de servicio.

Task 1.2

a. Sobreestimación

En DQN, el máximo usado en el target elige la acción con mayor estimación ruidosa. Si $\widehat{Q}(s,a)=Q(s,a)+\epsilon_a$, entonces normalmente:

$$E[\max_a \widehat{Q}(s,a)] \geq \max_a Q(s,a).$$

El máximo favorece por azar los errores positivos. Double DQN reduce este efecto al separar la red que elige la acción de la red que la evalúa.

Si se sobreestima *bloquear*, el sistema puede bloquear tráfico legítimo con demasiada frecuencia: interrumpe usuarios, aplicaciones y operaciones. Si se sobreestima *ignorar*, puede dejar pasar señales peligrosas: aumenta el riesgo de intrusiones no contenidas.

b. Targets de DQN y Double DQN

Para una transición (s,a,r,s') , el target de DQN estándar es:

$$y_{\text{DQN}} = r + \gamma(1-d)\max_{a'} Q(s',a';w^-).$$

Para Double DQN es:

$$y_{\text{DDQN}} = r + \gamma(1-d)Q(s',a';\arg\max_{a'} Q(s',a';w);w^-).$$

Aquí r es la recompensa, γ el descuento y d vale 1 si el episodio termina. w son los parámetros de la red en línea, que se actualiza con el entrenamiento. w^- son los parámetros de la red objetivo, que se copian desde w cada cierto número de pasos y dan mayor estabilidad.

El único cambio de implementación está en calcular el target: DQN usa el máximo de la red objetivo; Double DQN usa la red en línea para obtener el índice de la mejor acción y la red objetivo para obtener el valor de ese índice. La arquitectura, el buffer, la función de pérdida y la actualización periódica de la red objetivo pueden mantenerse iguales.

c. Estados donde el problema es más grave

La sobreestimación es más grave en estados ambiguos: tráfico borderline, señales incompletas o actividad que parece normal y maliciosa a la vez. En esos estados, los valores de las acciones son parecidos y el ruido de estimación puede cambiar cuál parece mejor. Eso puede producir bloqueos innecesarios o decisiones de ignorar peligrosas.

En estados claramente maliciosos, aislar o bloquear debería tener un valor mucho mayor que ignorar. Como la separación entre acciones es amplia, un error pequeño tiene menos probabilidad de cambiar la decisión. Double DQN sigue siendo útil, pero su beneficio operativo principal está en las decisiones inciertas.

Task 2.1 - Implementación desde cero

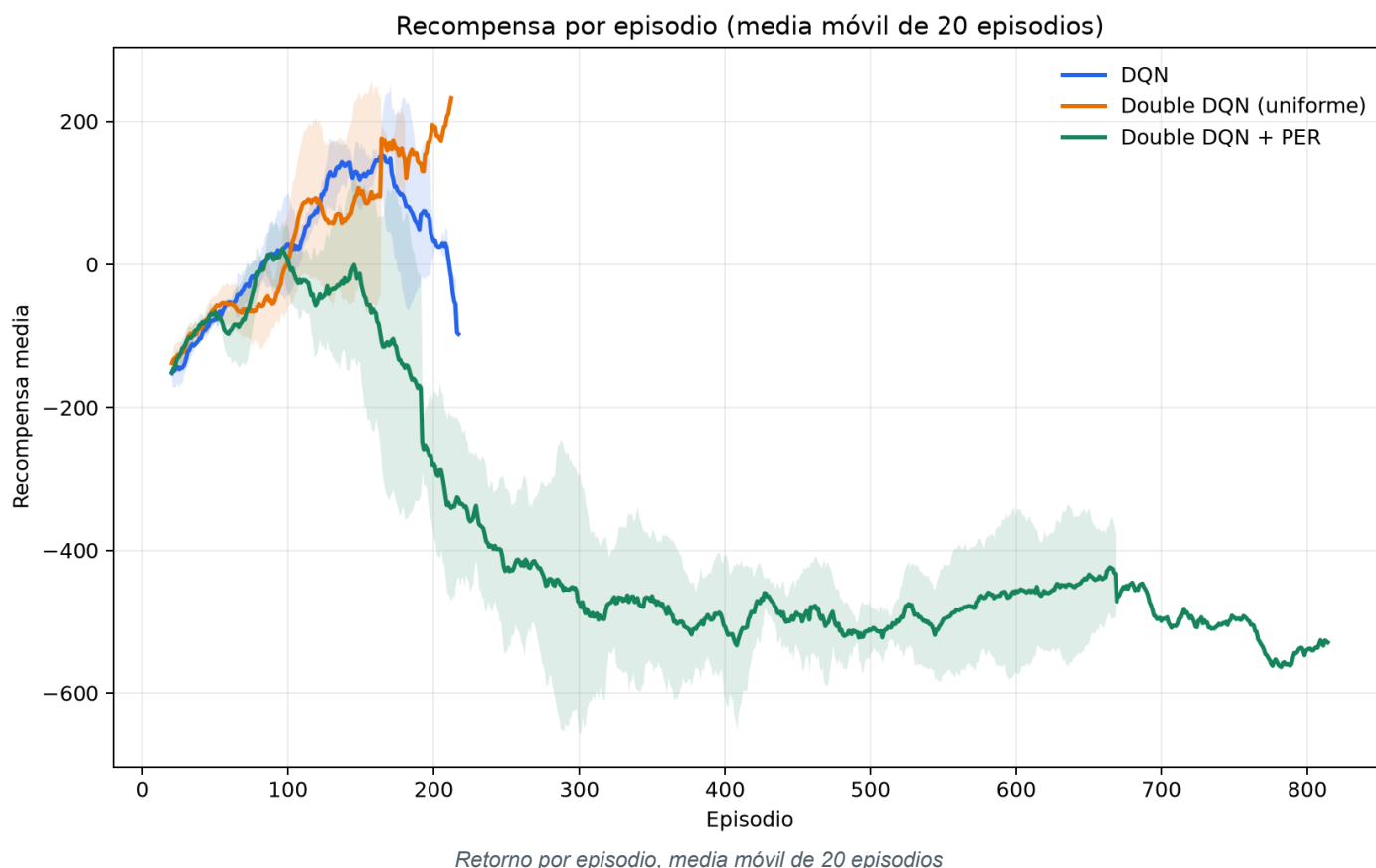
Se implementaron DQN, Double DQN y Double DQN con Prioritized Experience Replay (PER) en `task2\_dqn.py`, usando Gymnasium LunarLander-v2 y PyTorch, sin usar una librería de RL. La red es $8 \rightarrow 128 \text{ ReLU} \rightarrow 128 \text{ ReLU} \rightarrow 4$; el buffer tiene 50 000 transiciones, el minibatch 64 y la target network se copia cada 500 interacciones. ϵ baja linealmente de 1.0 a 0.05 en 10 000 pasos. Se usó $\gamma=0.99$, Adam con tasa 0.001 y pérdida Huber. DQN usa $\max_a Q(s, a)$; Double DQN elige $\arg\max$ con la red en línea y evalúa esa acción con la red target. PER usa $\alpha=0.6$, β lineal de 0.4 a 1.0 y pesos de importancia inversa normalizados.

Cada episodio completo registra (1) retorno total, (2) promedio de $\max_a Q(s, a)$ en los mismos 100 estados iniciales fijados por semilla, y (3) promedio de los targets de Bellman usados durante las actualizaciones de ese episodio. Los episodios incompletos al llegar al límite de pasos no se incluyen. Para reproducir:

```
python -m pip install -r requirements-task2.txt
python task2_dqn.py --max-steps 100000 --seeds 17 23 42 --output results/task2_metrics.csv
python plot_task2.py --input results/task2_metrics.csv --output-dir results
```

Configuración experimental: 100 000 interacciones por método y semilla, semillas 17, 23 y 42; las tres variantes usan las mismas semillas. Las bandas de las figuras muestran ± 1 desviación estándar entre semillas. LunarLander valida mecánicas algorítmicas, pero no es evidencia de desempeño de un detector IDS.

Task 2.2a - Recompensa, convergencia y recompensa final



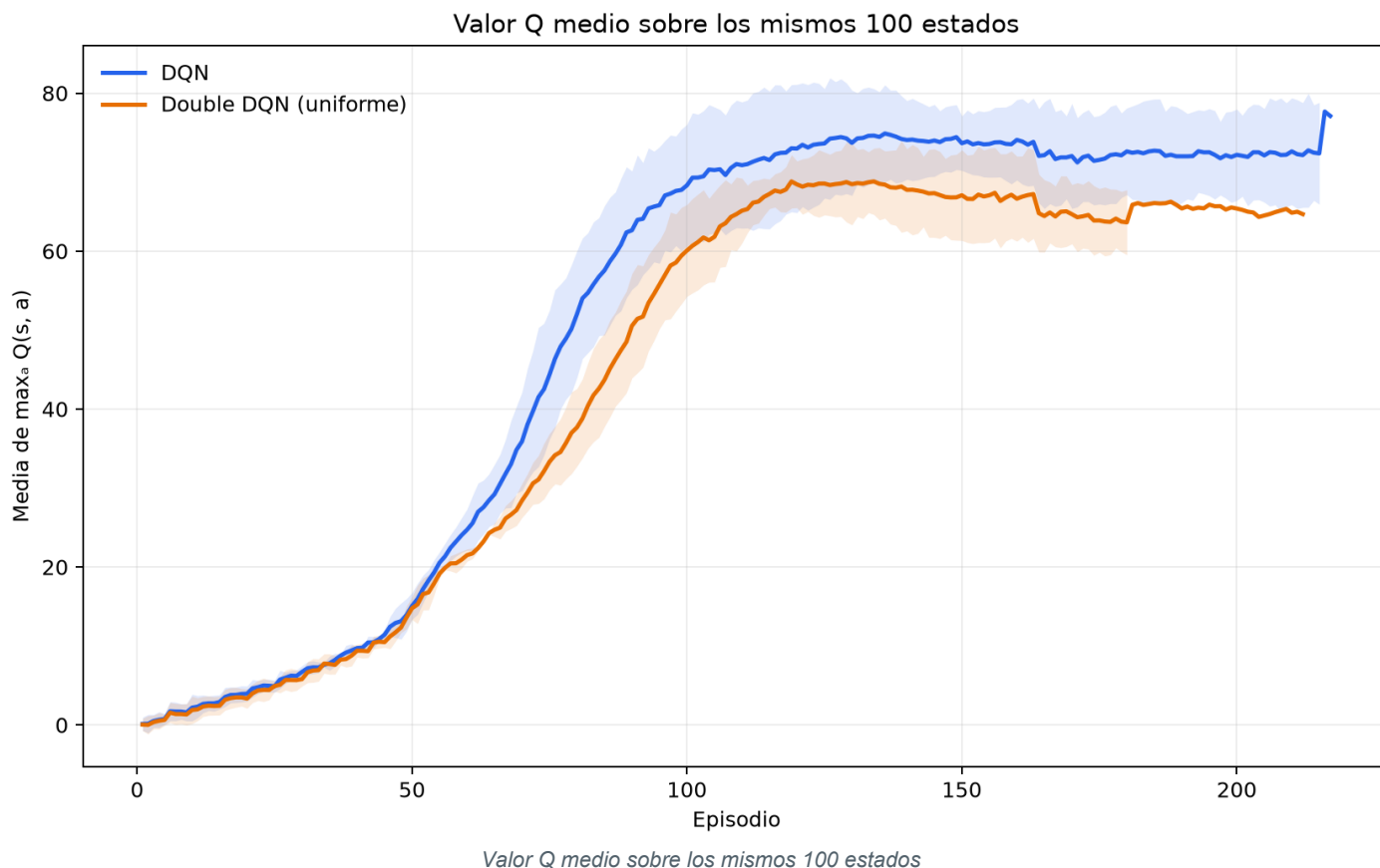
Método	Retorno medio últimos 20 episodios	Media últimos 100 episodios	Cruce de MA20 ≥ 200
DQN	10.1 ± 133.4	78.1 ± 23.4	1/3 semillas

Método	Retorno medio últimos 20 episodios	Media últimos 100 episodios	Cruce de MA20 ≥ 200
Double DQN uniforme	116.2 $\pm$ 159.1	85.1 $\pm$ 121.9	2/3 semillas
Double DQN + PER	-309.6 $\pm$ 271.5	-283.0 $\pm$ 308.0	0/3 semillas

En rapidez no hay un ganador independiente del criterio: DQN cruzó retorno móvil 0 en las 3 semillas; Double DQN lo hizo en 2/3. Al exigir retornos más altos, Double DQN cruzó MA20 ≥ 100 en 2/3 semillas a una mediana de 50 628 pasos entre las corridas que cruzaron, y MA20 ≥ 200 en 2/3 a una mediana de 87 711 pasos. DQN cruzó 100 en 3/3 (mediana 75 205 pasos) y 200 en 1/3 (81 805 pasos). Así, Double DQN llegó a recompensas altas en más semillas y tuvo mayor retorno final medio; DQN alcanzó mejoras modestas de manera más consistente. Ninguna corrida alcanzó el criterio convencional de LunarLander de promedio ≥ 200 en una ventana de 100 episodios.

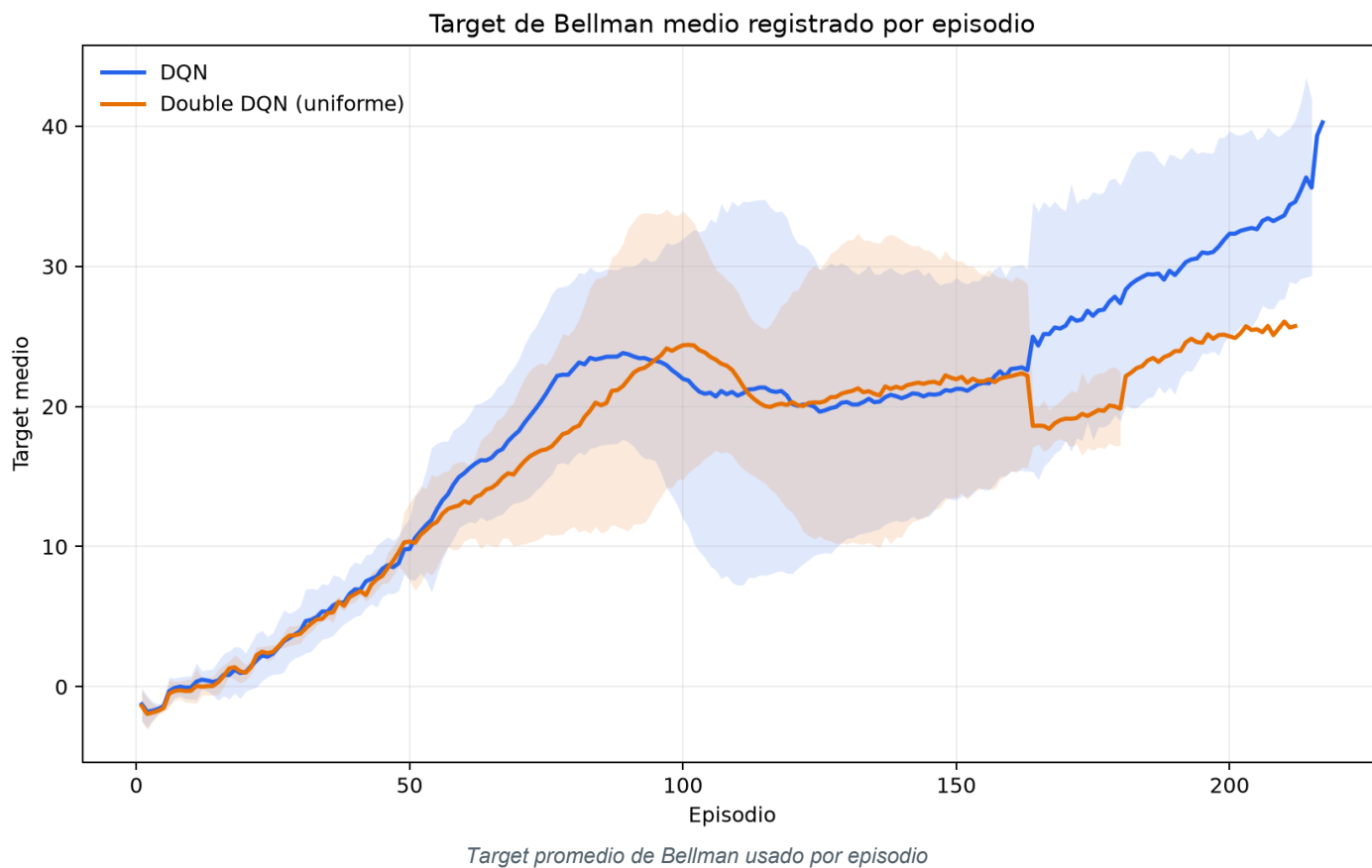
El resultado coincide parcialmente con la expectativa teórica: Double DQN mejora la recompensa final media y evita parte del exceso en Q, pero la alta desviación y el bajo número de semillas no permiten afirmar convergencia robusta ni una ventaja universal en velocidad.

Task 2.2b - Valores Q y señales de sobreestimación

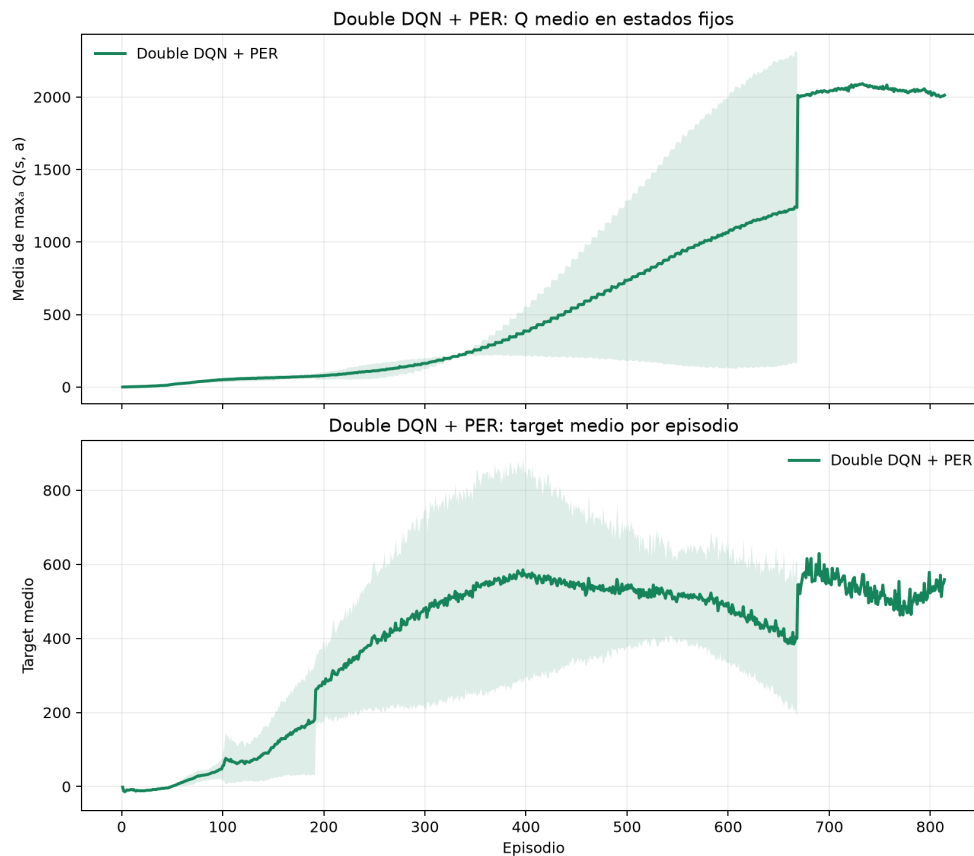


La métrica graficada es el promedio de $\max_a Q(s, a)$ sobre los mismos 100 estados de evaluación, medido al final de cada episodio. En los episodios emparejables por semilla, el Q de DQN superó al de Double DQN en **89.5%** de los casos; la diferencia media emparejada fue **+5.18** unidades Q. Al final de las corridas, Q medio fue **73.89 $\pm$ 5.27** para DQN y **65.82 $\pm$ 5.74** para Double DQN. En estas corridas sí se observa la dirección esperada, aunque la brecha es moderada y no significa que DQN sobreestime en cada estado o episodio.

La evidencia puede depender del entorno, del ruido de aproximación, de la sincronización de la target network y de la muestra pequeña de 100 estados iniciales. Una brecha no visible en otro entorno no refutaría por sí sola el sesgo teórico: otros errores y la distribución de estados pueden dominarlo.



En la figura de los métodos uniformes, DQN y Double DQN mantienen escalas comparables y se aprecia su diferencia sin que PER comprima el eje vertical. La figura siguiente separa PER para mostrar su inestabilidad a escala completa. Q y target son estimaciones internas; la recompensa del entorno permite comprobar si ese crecimiento corresponde a una política mejor.



Escalas Q y target de Double DQN con PER

Task 2.2c - Double DQN uniforme vs. Double DQN con PER

Con $\alpha=0.6$ y β creciente de 0.4 a 1.0, el PER sin límites de prioridad no aceleró la convergencia en esta configuración. Su retorno medio de los últimos 20 episodios fue -309.6 ± 271.5 , frente a 116.2 ± 159.1 con muestreo uniforme. La curva verde acompaña inicialmente a las demás, pero se deteriora desde aproximadamente los episodios 90-110 y luego cae con fuerza. Dos semillas alcanzaron 668 y 814 episodios en las mismas 100 000 transiciones porque la nave terminaba episodios muy cortos tras choques; contar más episodios aquí no significa aprender más rápido.

La divergencia coincide con Q y targets desmesurados: Q final medio en los estados fijados fue $857.81 \pm 1\,020.00$ y target final 279.17 ± 267.84 para PER, frente a 65.82 ± 5.74 y 24.49 ± 5.96 con Double DQN uniforme. En este experimento, la mayor diferencia aparece en la fase intermedia y tardía, después de que la curva inicial parecida se separa y el muestreo repetido por TD-error alimenta la inestabilidad. El resultado no demuestra que PER sea intrínsecamente malo; sí muestra que una prioridad proporcional directa puede amplificar errores en este proxy.

Task 2.2d - ¿Qué esperar en detección de intrusiones?

Task 1.1c calculó, para una proporción crítica $p<0.001$ y minibatch 64, una probabilidad aproximada de 93.8% de que el lote no contenga ningún evento crítico. Eso hace más fuerte la motivación para priorizar o garantizar ejemplos de intrusiones raras en el IDS que en LunarLander, donde no hay clases de ataques minoritarias equivalentes. Sin embargo, el experimento actual aporta una cautela empírica: PER basado solo en TD-error fue mucho peor que muestreo uniforme y generó Q/targets enormes en LunarLander. Un TD-error alto puede representar una observación ruidosa o un choque, no necesariamente un evento crítico útil.

Por ello, para el IDS sería más prometedor evaluar PER combinado con cuotas mínimas por clase o muestreo estratificado, límites de prioridad, pesos de importancia y una evaluación explícita de recall/falsos negativos por ataque. Debe compararse

contra replay uniforme con el mismo presupuesto y cortes temporales/por host; nuestros datos de LunarLander motivan ese control, pero no prueban que PER mejore el IDS.

Task 2.3a - Paper reciente: resumen técnico

Tellache, Mokhtari, Amara Korba y Ghamri-Doudane, “**Multi-agent Reinforcement Learning-based Network Intrusion Detection System**”, presentado en IEEE NOMS 2024. El trabajo aborda la clasificación multiclase de tráfico en CIC-IDS-2017, donde BENIGN domina y existen ataques con muy pocos ejemplos, además de la adaptación a patrones nuevos. Su arquitectura tiene agentes DQN de primer nivel, uno por tipo de ataque, y un agente decisor que combina sus salidas Q. Cada agente clasifica su ataque objetivo frente a otros ataques y tráfico normal. Para manejar el desbalance, los autores usan recompensas cost-sensitive y una pérdida de error cuadrático medio ponderada por muestra; los agentes tienen dos capas ocultas de 128 neuronas. Es DQN estándar, no Double DQN ni PER: el target conserva el máximo de Q, por lo que no se introduce la separación de selección y evaluación que ataca directamente la sobreestimación.

En CIC-IDS-2017 reportan 99% de exactitud, 99% de recall ponderado, F1 ponderado 0.99, FPR ponderada 0.16% y AUC global 0.92. También prueban adaptación: el recall de DoS Slowloris, omitido durante el entrenamiento inicial, pasa de 0.33 a 0.98 tras adaptar agentes con datos nuevos. Pero los promedios ocultan clases con soporte minúsculo: Infiltration tiene recall 0.43 con siete ejemplos de prueba; SQL Injection, recall 0.50 y F1 0.03 con cuatro; Heartbleed solo tiene dos. El trabajo aborda parcialmente la rareza mediante agentes especializados y recompensas ponderadas, y demuestra adaptación en una prueba de datos, pero no elimina el problema de escasez ni resuelve explícitamente la sobreestimación de DQN. Tampoco equivale a validación operativa en tráfico de producción.

Fuentes: [artículo completo en arXiv](#) · [registro de IEEE NOMS 2024](#) · [DOI 10.1109/NOMS59830.2024.10575541](#).

Task 2.3b - Dictamen técnico para el equipo directivo

Dictamen: recomendamos continuar únicamente con un piloto offline/shadow de **Double DQN con replay uniforme** como primera variante para el IDS. En el proxy, Double DQN terminó con retorno medio 116.2 ± 159.1 en los últimos 20 episodios frente a 10.1 ± 133.4 de DQN, y su Q medio en los estados fijos fue menor; esto respalda preferirlo ante la sobreestimación observada. No recomendamos desplegar todavía ni habilitar respuestas automáticas: tres semillas de 100 000 pasos no lograron el criterio de resolución y PER produjo retornos muy negativos junto con Q/targets explosivos. La rareza del Task 1 justifica investigar replay estratificado o PER con prioridades acotadas, pero primero se requiere validar con tráfico representativo, división temporal y por host, suficiente cobertura de ataques raros, recall y falsos negativos por clase, calibración de umbrales, comparación con un IDS supervisado, pruebas de latencia/deriva y operación shadow con revisión humana y rollback. La evidencia de LunarLander sirve para escoger qué variante estudiar, no para afirmar seguridad del detector en producción.